

Autonomous Mid-Air Reorientation for Jumping Robots

Neel Tripathi

ME 200P

Professor Hawkes

Abstract

This project presents the development of an autonomous flight-control system for a jumping robot capable of reorienting itself during ballistic flight. The system uses an ESP32-C3 microcontroller and a WT901 inertial measurement unit (IMU) to detect launch, estimate the robot's trajectory, and command two reaction wheels to align the robot's foot with its velocity vector prior to landing. A state-machine-based control architecture separates launch detection, velocity estimation, and ballistic attitude control. Body orientation is propagated using gyroscope integration, while the velocity vector is estimated from acceleration measurements during launch and gravity integration during flight. Preliminary testing demonstrated reliable launch detection and attitude control responses, although additional work is required to validate the state estimations, tune the PD orientation controller, and test high-speed launches. This system will be named Cooper, after its future goal of recuperating energy.

1. Introduction

Jumping robots are emerging as a promising class of extraterrestrial exploration robots due to their ability to bypass what would be inaccessible to traditional wheeled or legged rovers, including obstacles several times their body height, discontinuous terrain, and cluttered environments. By storing energy and releasing it rapidly, jumping systems can achieve high peak power outputs while remaining lightweight and mechanically simple.

One current difficulty in using jumping robots for exploration, however, is their energy efficiency metric. After a launch, the stored potential energy is converted into kinetic energy, and the robot takes off. After the full jump trajectory is achieved, the robot impacts the ground and much of the kinetic energy generated during launch is dissipated. Recovering even a fraction of this impact energy could substantially reduce the energy required for subsequent jumps, improving the overall system efficiency and reducing the total locomotive energy expenditure. Because impact energy recovery mechanisms are most effective when the robot lands in a controlled orientation, reliable mid-air attitude control becomes a critical enabling technology. Poor landing alignment can reduce energy-recovery efficiency or damage the recovery mechanism entirely. While significant research has focused on maximizing jump height and efficiency, successful landing remains a major challenge. Small orientation errors during flight can

result in unstable landings, tip-overs, or structural damage, and because jumping robots typically spend only a short time in contact with the ground, they have limited opportunities to correct their trajectory once airborne.

This project aims to develop an embedded control system capable of autonomously reorienting a jumping robot to align with its predicted landing trajectory in ballistic flight in order to maximize the energy recoupment upon impact. Unlike systems that rely on external motion capture or vision-based tracking, this approach estimates the vehicle state using only onboard inertial sensing and computation, enabling a compact and self-contained platform capable of autonomous mid-air reorientation. In this system, an IMU measures body orientation and angular velocity, while an ESP32-C3 executes state estimation and control algorithms in real time. Two motors generate corrective torques that rotate the robot during the ballistic phase, using two attached reaction wheels.

2. Hardware Design

2.1 Design Criteria

The objective of the jumper design was to maximize jump height while maintaining a maximum control authority for mid-air reorientation. Because the jumping mechanism stores a fixed amount of energy, reducing the mass of the robot body increases the velocity that can be achieved at takeoff, and therefore improves jump performance. This means that lightweight materials such as carbon fiber and 3D-printed polymers like PLA or PPA-CF were used wherever possible.

In addition to minimizing overall mass, the ratio of body mass to foot mass was also considered. The jumping mechanism relies on momentum transfer between the body and the foot during launch (Figure 1). Increasing the body-to-foot mass ratio improves the effectiveness of this momentum exchange, allowing a greater fraction of the stored energy to be transferred into upward motion. As a result, the foot will be designed to be as lightweight as possible while maintaining sufficient strength for repeated jumping and landing cycles. This prototype is being designed to fit in a confined centralized space, to ensure that the entire reorienting mechanism will be able to serve as body mass in the final jumping robot.

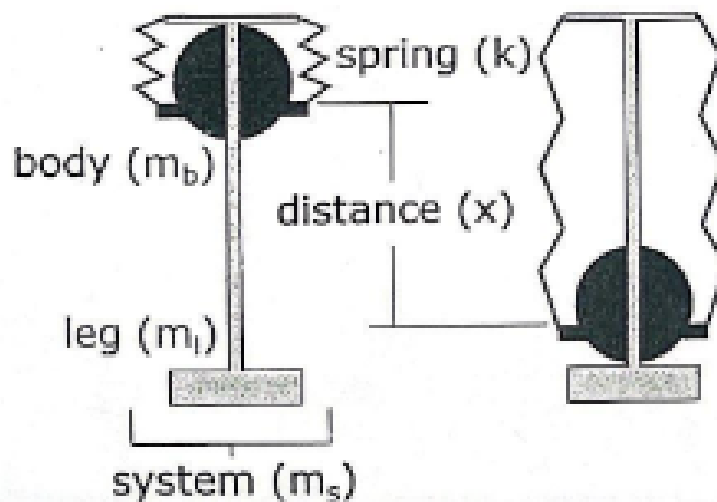


Figure 1: Simple jumper model, with labeled body and leg masses. Body mass accelerates upwards and undergoes a momentum transfer after inelastically colliding with the leg mass

A final design consideration was controllability during flight. Components were positioned to concentrate mass near the center of the vehicle and to align the center of mass with the reaction-wheel assembly. This reduces rotational inertia and minimizes dynamic coupling, allowing the reaction-wheel actuators to more effectively control vehicle orientation during ballistic flight.

2.2 Robot Structure

The robot is built around a 1 mm thick 10 mm x 10 mm square carbon fiber tube. The square profile was selected to simplify integration with the energy recoupment mechanism being developed by other researchers. The robot body was designed to consist of a small number of 3D-printed components requiring minimal post-processing, allowing rapid iteration during development and easy replacement of damaged parts. A 3D printed frame was designed to clamp onto the carbon fiber shaft while compressing a thin sheet of Dycem rubber to maximize the friction force. The clamping part is designed to compress the rubber by 0.1 mm, so in addition to the high surface friction coefficient of Dycem, it also greatly increases the normal force felt by the clamping surface. A smaller 3D printed bracket is first screwed onto two VertIQ motors using the two bolt holes closest to the center of the part on each motor, and then the carbon fiber tube is clamped between the bracket and the frame, secured using the two remaining bolt holes on each motor. A cross section of this assembly is shown in Figure 2. A diagonal cutout was made on the frame poka-yoke (Figure 2 right), as without this, the interference with the edge of the bracket would have prevented assembly.

This assembly can be slid along the carbon fiber tube before clamping to ensure that the center of mass of the entire system is centered on the reaction wheels, as this makes the control simpler by reducing coupling between translational and rotational dynamics of the body in motion, and also minimizes the effects of gravitational torque in flight.

The motors have reaction wheels mounted onto their rotors. These motors are mounted at 90 degrees to each other, parallel to the faces of the carbon fiber tube (Figure 2) so that the pitch and roll axes can have independent control. Firing one motor causes a torque about the pitch axis, while firing the other causes a torque about the roll axis. These two rotational degrees of freedom are sufficient to align the landing foot with the in-flight velocity vector, and since yaw orientation does not significantly affect landing performance, only two rotational degrees of freedom were actively controlled.

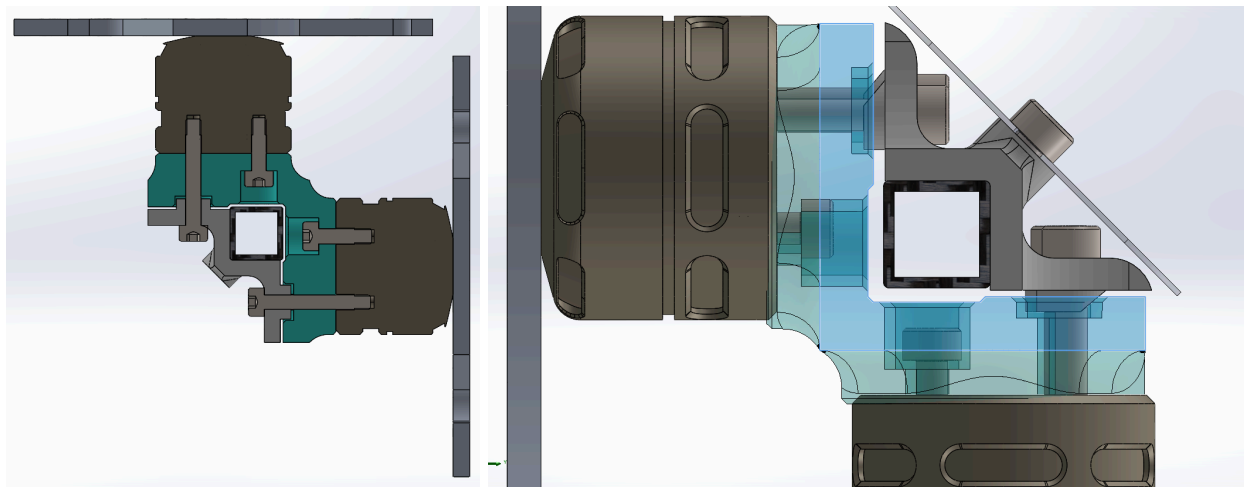


Figure 2: Cross section of tube clamping arrangement (left). Motor bracket is blue, frame is light gray. Translucent bracket showing frame poka-yoke (right).

An electronics proto-board is mounted to the frame using heat set threaded inserts, with small o-rings between the frame and the board to minimize vibrations. The holes in the corners of the protoboard support M2 screws. A battery support is also built into the frame, with a ledge and a strap to locate and to secure the battery respectively.

The reaction wheels were designed to be both 3D printable and machinable out of metal. When 3D printed, bolts can be installed to increase the moment of inertia. The metal reaction wheels have a moment of inertia of $47357.29 \text{ grams} \cdot \text{mm}^2$ about their rotational axis when manufactured out of aluminum, with a total mass of 45.51 grams. Aluminum was selected as the material of choice, as the stress in a flywheel scales with density rather than mass, and so the performance index to maximize is allowable stress

divided by density. Searching this metric on an Ashby chart shows that Aluminum is a good choice for its low density and easy machinability.

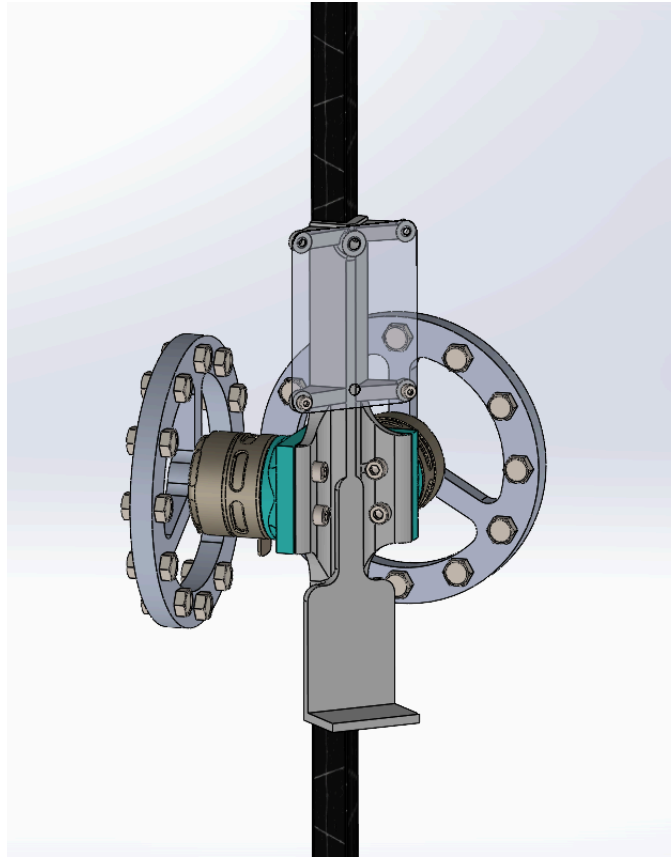


Figure 3: Jumper frame assembly. An electronics protoboard is also seen (transparent)

A 3D printed cap was also installed at the end of the jumper, to ensure that a direct hard landing does not damage the carbon fiber tube.

2.3 Mechatronics Design

Controls and state estimations are calculated by a Seeed Studio Xiao ESP32-C3 microcontroller. This was chosen due to its extremely small size, built-in WiFi connection capability, and computational power. The integrated WiFi functionality of the ESP32 was also used for wireless telemetry during testing, eliminating the need for a separate communications module. Vehicle state sensing is provided by a WT901 nine-degree-of-freedom inertial measurement unit, which supplies accelerometer, gyroscope, and orientation measurements required for launch detection and attitude estimation. Control is provided by two VertIQ 23-06 G1 220 kV motors, which are able to spin at a maximum speed of 199 radians per second, and are able to produce 0.065 Nm of torque. This can accelerate the reaction wheels at 1372 rad/s^2 . These motors

include integrated motor controllers and an accompanying serial communication interface library, eliminating the need for separate motor driver electronics. This significantly simplifies the control circuitry while reducing both the wiring complexity and the overall system footprint.



Figure 4: Prototype of control circuit (left) and condensed protoboard circuit (right)

The robot senses, computes, and actuates with careful consideration taken to centralize heavy components near the center of the jumper and to minimize the volume taken by wiring. Because of this, the microcontroller, IMU, and reset button were all permanently soldered to a protoboard that was mounted directly to the frame (Figure 3). Consolidating the electronics into a single assembly improved reliability during repeated launch testing and minimized the risk of loose wiring causing hard to diagnose errors. Additionally, the completed electronics package was designed such that all control hardware could be removed from the frame as a single assembly, allowing for easy replacement in the case of structural failure or design iteration. This protoboard was mounted at a 45 degree angle to the sides of the square carbon fiber tube to maintain symmetry and to minimize the jumper hardware's footprint, resulting in a rotated coordinate system for the motors relative to the jumper's sensory frame of reference. The controller accounts for this by performing a coordinate transformation that combines the motor inputs to generate the desired control torques about the correct body axes.

Power is supplied by an 11.1 V, 550 mAh lithium-polymer battery mounted directly to the frame. The battery was selected to provide sufficient discharge capacity for both the jumping mechanism and the reaction-wheel control system while maintaining a low

system mass. The battery output is distributed in parallel to the two motors and a voltage-regulation circuit. A 5 V regulator supplies power to the microcontroller and IMU, while the motors operate directly from the battery voltage. Locating the battery near the vehicle center also reduces its contribution to the rotational inertia of the jumper. Power distribution is handled by a custom module consisting of interconnected XT-style connectors, allowing the battery, motors, and voltage regulator to be connected through a common interface. A power switch was placed in series with the battery output to provide a reliable means of disconnecting power during testing and maintenance. The use of XT-style connectors throughout the system makes the power distribution system modular and allows the battery, motors, and voltage regulator to be replaced independently during testing and maintenance.

3. Software Design

3.1 Software Overview

The system's firmware autonomously detects launch, estimates the vehicle state during flight, and uses motor torques to align the landing foot with the direction of travel. The controller receives accelerometer, gyroscope, and orientation measurements from the IMU and outputs velocity commands to the two reaction-wheel motors. This architecture allows the jumper to estimate its future landing orientation using only onboard sensing and computation, without requiring external tracking systems. Estimating the vehicle trajectory is particularly challenging because accelerometers measure specific force rather than absolute acceleration. During ballistic flight, the jumper will experience freefall and receive little information about the jumper's translational velocity. This controller must rely solely on the initial launch conditions to propagate the trajectory forward throughout the rest of the jump using a ballistic flight model.

All software was implemented in C++ and executed on the ESP32-C3. The controller operates as a state machine with two operating modes: pre-launch and ballistic flight. Separating these modes allows different estimation and control strategies to be applied before and after takeoff.

During the pre-launch phase, the controller estimates the gravity vector using IMU orientation measurements while monitoring accelerometer data for a launch event. Launch is identified by a sustained increase in the measured acceleration for a period of time, followed by a return to low acceleration. This approach was originally developed to minimize false triggers while testing the prototype circuit during bench testing, but was extremely effective at filtering sensor noise and preventing the controller from generating unwanted destabilizing torques during the launch. During the launch impulse, the measured acceleration is transformed into the world frame using the

estimated jumper orientation and integrated to estimate the initial launch velocity and trajectory angle. These estimates are then used to initialize the ballistic flight model.

After launch is detected, the controller propagates the velocity estimate by integrating the gravitational acceleration vector in the world frame. Simultaneously, gyroscope measurements are integrated to estimate the vehicle orientation using the orientation measured prior to launch as the initial condition. The jumper orientation is handled in quaternions to avoid singularity issues common when using Euler angle orientation representations.

The jumper orientation is controlled with a PD controller to align the robot's foot axis with the estimated velocity vector. Aligning the foot axis with the real-time velocity vector increases the likelihood of a stable landing and reduces the chances of tipping on impact, and allows the jumper to adapt to variations in landing elevation. An error quaternion is calculated between the current orientation and the desired velocity aligned orientation, and is converted into rotational error components. The error components are then sent to the reaction wheels, which generate corrective torques to reorient the jumper in flight.

A wireless telemetry system was also developed to assist in debugging and evaluating the accuracy of the state-estimation and control algorithms being conducted by the controller. The Xiao ESP32-C3 hosts a WiFi server that streams the current state estimates and controller outputs to an external computer running a python script. This script visualizes the estimated 3D orientation in real time and logs the telemetry data to a file to analyze after the flight. This capability allows the controller estimates to be compared with external slow-motion video so that the jumper's launch detection, velocity vector estimation, and orientation control performances can be validated.

3.3 Optimized Control Insights

Several control strategies were investigated in simulation prior to implementation on the physical platform. These included a conventional Linear Quadratic Regulator (LQR), a trajectory-optimized tracking LQR (TLQR), and a reaction-wheel braking strategy in which angular momentum stored in the flywheel is rapidly transferred to the robot body by braking near the end of flight.

Simulations conducted showed that a braking controller generally required the least control effort because it relied primarily on angular momentum stored in the reaction wheel rather than continuous motor actuation. However, the performance of this approach was highly sensitive to launch-condition estimation and braking timing, making it less robust to uncertainty. The standard LQR controller provided a useful baseline but tended to apply large corrective torques immediately after launch because

it regulated directly to a fixed landing orientation. This resulted in higher control effort than either the braking or trajectory-tracking approaches. Finally, a tracking LQR controller produced the most accurate landing orientations by following an optimized attitude trajectory throughout the jump, but was substantially more complex and intensive to implement in real time.

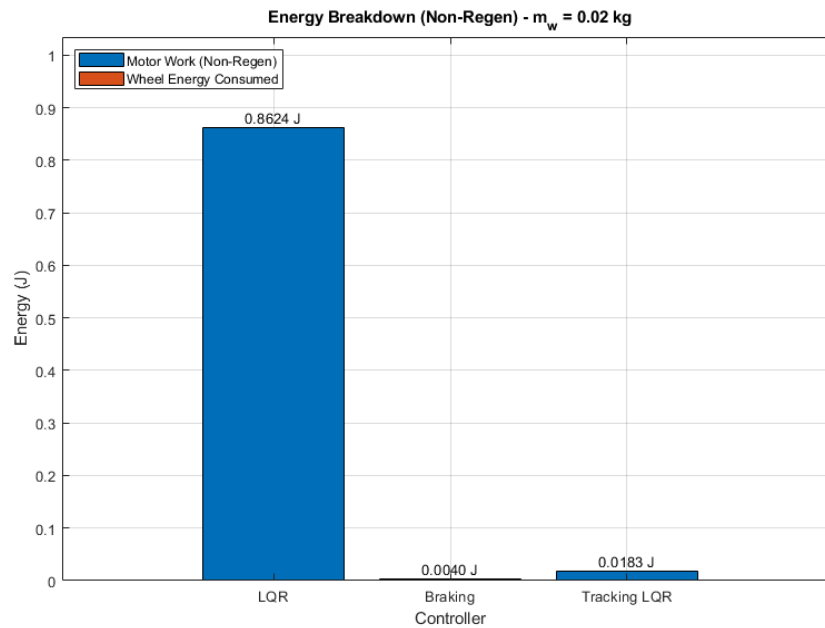


Figure 5: Relative energy expenditure by each control strategy

Although these advanced control strategies were not implemented on the prototype hardware, they did confirm that reaction wheel attitude control is in fact able to reorient the vehicle within the available flight time. They also demonstrated that accurate estimation of the launch conditions is critical as the translational trajectory becomes unobservable during ballistic flight.

The increase in control effort required to run a slightly less-optimal controller was negligible compared to the energy gained by a successful recoup, so the relatively simple and robust PD controller was finally selected.

4. Testing and Validation

Several tests were performed throughout development to validate sensing, estimation, communication, and control subsystems.

4.1 IMU Testing

The WT901 IMU was first validated through serial telemetry to verify proper communication and physically realistic accelerometer, gyroscope, and orientation outputs. Measurements were monitored using the calibration software while manually rotating and translating the sensor to confirm consistency with expected motion.

4.2 Launch Detection Testing

First, benchtop tests were performed to ensure that a manual rapid acceleration of the sensor was detectable by the system. After this was confirmed and the jumper frame was assembled, manually thrown launch tests were performed to ensure that the algorithm accurately detected the transition between launch and ballistic flight. The code had to be edited several times to get rid of edge cases. The two bugs encountered were the jumper initializing with spinning reaction wheels at idle and attempting to control its orientation while a launch impulse was still being applied.

4.3 Telemetry Verification

The wireless telemetry system successfully streamed orientation estimates, velocity estimates, attitude errors, and motor commands to an external computer in real time. One problem encountered while benchtop testing was an occasional non-updating velocity estimate. When this happened, the orientation control was still able to generate wheel torques to align the jumper body to the desired jumper foot orientation that was observed at launch, but the velocity vector did not seem to be integrating the acceleration caused by gravity that had been last measured. This may have been a quirk only experienced due to the imperfect manually simulated launch, as in real flight testing, the telemetry data showed that the jumper was seemingly able to update its velocity vector and reorient for alignment.

5. Results and Discussion

The completed system successfully demonstrated all major components of the intended control architecture. The IMU provided stable orientation and acceleration measurements, the launch-detection state machine reliably identified transitions between launch and ballistic flight, and the wireless telemetry system successfully streamed real-time state information throughout testing.

After correcting the launch-detection logic to wait for a drop back to low acceleration detected, the controller consistently remained inactive while the jumper was stationary or experiencing the launch impulse. This eliminated the earlier issue in which the

reaction wheels could begin spinning while the robot was idle or attempt to generate corrective torques before the jumper had entered free flight.

One issue observed during bench-top testing was a velocity estimate that failed to update after launch. In these cases, the controller continued to regulate the vehicle toward the launch velocity vector rather than the continuously updated velocity direction. However, this behavior was not always observed during jumper flight testing, where the velocity estimate appeared to evolve normally under gravitational acceleration, with just a slightly delayed and overshoot attitude correction observed most of the time (Figure 6). Additional testing is required to determine whether this issue is related to the bench-test manual acceleration procedure or an issue in the velocity propagation or gravity compensation logic.



Figure 6: Working midair trajectory control, with some final angle overshoot. Edited frames together from Example_Vid.mov (appendix)

The main remaining challenge after correcting any software bugs will be to finish tuning the jumper controller, potentially using a dynamics simulation to more rapidly test parameters, validating the telemetry data of a successful test against external video footage, and quantifying the accuracy of the final orientation with larger scale jumps.

One area of weakness may also be the computational load being placed on the singular ESP32-C3 microcontroller. As it is simultaneously reading sensor updates, estimating translational velocity, vehicle orientation, and angular velocity in real time, calculating control signals, and sending telemetry over WiFi, it is possible that this is reducing the accuracy of the various computations being performed. While this was not an issue

during individual subsystem testing, splitting each task across multiple networked microcontrollers may improve control response and accuracy.

6. Conclusion

This project demonstrated the feasibility of autonomous mid-air reorientation for a jumping robot using only onboard sensing and computation. A complete hardware and software system was developed around an ESP32-C3 microcontroller, WT901 IMU, and quaternion-based attitude controller to detect launch, estimate trajectory, and command corrective torques during ballistic flight. The project successfully establishes a platform with the core sensing, estimation, and control hardware and software necessary for autonomous aerial reorientation in a compact and self contained design.

While additional testing and controller refinement are required to fully quantify landing accuracy and robustness, the project achieved its objective of creating an autonomous reorientation system for a jumping robot. Inertial sensing, reaction-wheel actuation, and embedded state estimation can be integrated to enable controlled orientation correction during ballistic flight. This capability is an important step towards designing jumping robots that can reliably recover landing energy and achieve the efficiency and mobility required for extraterrestrial exploration in complex terrain.

Appendix

Repository Map

(https://github.com/NLT319/Reorienting_Jumper)

- Reorienting_Jumper
 - Archive/
 - ESP32C3/
 - ESP32C3_IMU/
 - DESIGN_DOC.md
 - ESP32C3_IMU/
 - DESIGN.md
 - include
 - README
 - lib
 - README
 - JY901/
 - JY901.cpp
 - JY901.h
 - platformio.ini
 - src
 - main.cpp
 - PD_controlled.cpp
 - pd_controlled.h
 - telemetry.cpp
 - telemetry.h
 - telemetry_log.csv
 - test
 - README
 - tools
 - telemetry_client.py
 - MATLAB_Sim/
 - Control_Scheme_Comparison/
 - quaternionCubePDControl.m
 - Quaternion_PD/
 - Media/
 - braking_controller.png
 - ControllerComparison.png
 - Example_Vid.mov
 - Low_Contrast_Vid.mov
 - Out_Of_Frame_Vid.mov
 - IMU_Test.mov
 - Jump_Trajectory_Control.png
 - lqr_controller.png
 - Quaternion_PD.mp4
 - trajopt_tlqr_control.png
 - WorkingJumperSim.mp4
 - README.md